

Especificación de Requisitos de Software (ERS) para el Ecosistema Digital MaduraApp

Sistema de Análisis de Madurez Agrícola Basado en Inteligencia Artificial, Cómputo en la Nube y Visión Computacional

1. Introducción

La gestión eficiente de los productos perecederos representa uno de los desafíos más críticos y complejos de la cadena de suministro agroalimentaria global contemporánea. De acuerdo con estimaciones recientes, aproximadamente un tercio de los alimentos producidos para el consumo humano se pierde o desperdicia a nivel mundial, lo que equivale a unos 1.300 millones de toneladas anuales.¹ En el sector minorista y doméstico de los países industrializados, una proporción significativa de este desperdicio ocurre debido a estándares de calidad estética exigentes y a la falta de conocimiento técnico por parte de los consumidores y empleados sobre el estado real de madurez de los productos.² MaduraApp surge como una respuesta tecnológica de vanguardia ante esta problemática, integrando algoritmos de visión artificial de última generación y arquitecturas de microservicios distribuidos para transformar la percepción empírica de la calidad en un diagnóstico basado en datos objetivos.³

1.1. Propósito

El propósito fundamental de este documento es definir de manera exhaustiva y técnica la Especificación de Requisitos de Software (ERS) para el desarrollo de MaduraApp, bajo el estándar internacional IEEE 830-1998.³ Este informe está diseñado para servir como la piedra angular técnica para el equipo de desarrollo, arquitectos de soluciones, científicos de datos y evaluadores académicos de la institución Duoc UC.³ A través de esta especificación, se busca garantizar que los atributos de calidad, las funcionalidades de visión artificial y la arquitectura cloud se integren de forma coherente para satisfacer la necesidad de reducir el desperdicio alimentario y optimizar la selección de productos climatéricos en entornos agrícolas y de puntos de venta.³

1.2. Ámbito del Sistema

El sistema objeto de esta especificación se denomina MaduraApp, una plataforma móvil nativa que utiliza Inteligencia Artificial para el análisis de madurez post-cosecha.³ El sistema operará específicamente sobre frutos climatéricos, definidos biológicamente como aquellos que experimentan un aumento súbito en su tasa respiratoria y en la producción autocatalítica de etileno después de ser recolectados, lo que permite su maduración fuera de la planta.⁴

Componente	Descripción y Alcance Técnico
Frontend Móvil	Aplicación nativa desarrollada en Kotlin para Android, utilizando la librería Jetpack CameraX para la gestión avanzada del hardware óptico y el pre-procesamiento de imágenes en el dispositivo. ³

Componente	Descripción y Alcance Técnico
Backend de Inferencia	Microservicio desarrollado en Python utilizando el framework FastAPI, encargado de orquestar la recepción de imágenes, la ejecución del modelo de IA y la devolución de diagnósticos en tiempo real. ³
Núcleo de IA	Modelo de visión artificial basado en la arquitectura YOLO26n (You Only Look Once, versión enero 2026, Ultralytics), optimizado para la detección de objetos y clasificación multiclase de estados de madurez. Se utiliza la variante Nano para compatibilidad con infraestructura cloud de nivel gratuito (CPU, <512MB RAM). ³
Infraestructura Cloud	Despliegue distribuido en servicios de Amazon Web Services (AWS) o Render, utilizando modelos de servicio PaaS e IaaS para garantizar escalabilidad y alta disponibilidad. ³

Lo que el sistema hará:

- Capturar imágenes de alta resolución de frutas seleccionadas mediante dispositivos móviles.³
- Detectar y localizar automáticamente la fruta dentro del encuadre fotográfico, ignorando el ruido visual del entorno.³
- Clasificar el estado de madurez de 4 tipos de productos climatéricos (Aguacate, Plátano, Tomate y Mango) en categorías de "Inmaduro", "Óptimo" y "Sobre-maduro".³
- Proporcionar recomendaciones de consumo y almacenamiento basadas en el diagnóstico.³
- Mantener un historial persistente de consultas para el seguimiento de calidad por parte del usuario.³

Lo que el sistema no hará:

- Análisis de frutas no climatéricas en la versión MVP (ej. cítricos, uvas).³
- Ejecución de inferencia de visión artificial en modo offline. El historial de escaneos previos puede consultarse sin conexión gracias al cache local implementado con Room (SQLite embebido en Android).³
- Medición de parámetros químicos internos mediante sensores invasivos (ej. grados Brix o pH directo).³

1.3. Definiciones, Acrónimos y Abreviaturas

- **IEEE 830:** Estándar para la especificación de requisitos de software que asegura la completitud y trazabilidad del desarrollo.³
- **Climatérico:** Propiedad fisiológica de ciertos frutos de madurar tras la cosecha mediante la producción de etileno.⁴
- **YOLO26n:** Modelo de red neuronal convolucional de la familia YOLO (You Only Look Once, versión enero 2026, Ultralytics), diseñado para la detección rápida y precisa de objetos en imágenes. Es un 43% más veloz en CPU que versiones anteriores, sin Non-Maximum Suppression (NMS) ni Distribution Focal Loss (DFL).⁹
- **FastAPI:** Framework de alto rendimiento para Python que permite la creación de APIs asíncronas seguras y rápidas.⁸

- **Kotlin:** Lenguaje de programación oficial para el desarrollo de Android, enfocado en la seguridad de tipos y la concisión.¹⁶
- **Restricción de Diseño:** Limitación técnica impuesta al desarrollo para garantizar la compatibilidad con el entorno de ejecución o el presupuesto.³

1.4. Referencias

Este documento se fundamenta en las siguientes fuentes y normativas:

- Estándar IEEE 830-1998 para ERS.³
- Documentación técnica de MaduraApp (Claudio Aro, 2024).³
- Guía de evaluación TPY1101 de Duoc UC sobre calidad y seguridad de la información.³
- Manual de Ética e Integridad y Política de Seguridad de la Información Duoc UC.¹⁷
- Estándares biológicos de madurez de la USDA y la FAO.¹

1.5. Visión General del Documento

El presente reporte se estructura en secciones lógicas que facilitan la comprensión del sistema tanto desde una perspectiva de negocio como técnica. La sección de Descripción General aborda el contexto del producto, sus funciones principales y las restricciones del entorno. La sección de Requisitos Específicos detalla las interfaces, funciones de software y atributos de calidad exigidos. Posteriormente, se incluye un análisis exhaustivo de la arquitectura tecnológica (Android, Python, Cloud) y de la base biológica del proyecto (frutas climatéricas), concluyendo con una justificación metodológica basada en Scrum y CRISP-DM para garantizar la viabilidad del desarrollo.³

2. Descripción General

2.1. Perspectiva del Producto

MaduraApp no es una aplicación aislada, sino el componente cliente de un ecosistema de agricultura inteligente (AgroTech) que conecta la percepción física del producto en el punto de venta con la capacidad de procesamiento de la nube.³ El sistema opera bajo un modelo de arquitectura distribuida donde el procesamiento pesado de visión artificial se delega a servidores escalables, permitiendo que dispositivos móviles de gama media puedan acceder a diagnósticos complejos sin agotar sus recursos locales de batería y memoria.³

El sistema interactúa con los siguientes elementos externos:

- **Hardware Óptico del Dispositivo:** Acceso directo a la cámara nativa mediante APIs de bajo nivel para capturar datos brutos de imagen.³
- **Infraestructura de Red:** Dependencia de redes 4G/5G o Wi-Fi para la transferencia de datos binarios (imágenes) y estructurados (JSON).³
- **Motores de Almacenamiento:** Conexión con bases de datos relacionales (PostgreSQL 16 en producción; SQLite conaiosqlite en desarrollo local). El ORM utilizado es SQLAlchemy async con Alembic para gestión de migraciones.³
- **Servicios Cloud:** Alojamiento de la lógica de negocio y del modelo de inferencia en plataformas como AWS o Render, optimizando la latencia mediante el despliegue geográfico cercano al usuario.³

2.2. Funciones del Producto

Las funciones del sistema se han diseñado para mitigar la asimetría de información en la compra de productos frescos. El proceso de negocio afectado comienza en la estantería del supermercado o en la bodega agrícola, donde la incertidumbre sobre la calidad interna de frutas como el mango o el aguacate genera desperdicio por rechazo o compra fallida.²

Función Principal	Mecanismo de Acción y Valor Agregado
Captura Fotográfica Inteligente	Utiliza CameraX para guiar al usuario en la toma de una foto nítida, gestionando el enfoque y la luz ambiental para optimizar la entrada del modelo de IA. ³
Detección y Segmentación Visual	Localiza la fruta dentro del entorno, discriminando entre el producto y el ruido del fondo (manos, estantes, etiquetas) para evitar falsos positivos. ³
Análisis de Madurez Multiclase	Evalúa patrones cromáticos y morfológicos específicos de cada especie para determinar el estado fisiológico actual del producto. ³
Entrega de Diagnóstico Semafórico	Presenta un resultado visual inmediato (Verde, Amarillo, Rojo) que permite al usuario tomar una decisión de compra en menos de 5 segundos. ³
Sugerencias de Almacenamiento	Proporciona consejos basados en datos (ej. "madurar en bolsa de papel" o "refrigerar inmediatamente") para extender la vida útil del producto. ³

2.3. Características de los Usuarios

El sistema identifica dos perfiles de usuario críticos que determinan el diseño de la interfaz y la robustez de la lógica:

- **Consumidor Final (B2C):** Usuarios con nivel educacional variado que buscan una herramienta de apoyo rápido durante sus compras. Requieren una interfaz de "fricción cero" con un máximo de dos interacciones para obtener el resultado. Su experiencia técnica suele ser básica, limitada al uso general de smartphones.³
- **Operador de Retail o Productor Agrícola (B2B):** Usuarios con mayor conocimiento del producto pero que necesitan herramientas de auditoría de calidad rápidas para la gestión de inventarios. Este perfil utiliza la aplicación para decidir qué lotes de fruta deben recibir descuentos por madurez avanzada o ser procesados para subproductos.³

2.4. Restricciones de Diseño y Desarrollo

El desarrollo de MaduraApp está sujeto a limitaciones técnicas y normativas estrictas para asegurar su viabilidad académica y profesional:

- **Lenguajes Obligatorios:** El uso de Kotlin para el desarrollo nativo en Android asegura un rendimiento óptimo en la gestión de hilos y cámara. Python es indispensable en el backend por su ecosistema de librerías de visión artificial (PyTorch, Ultralytics).³
- **Capacidad de Cómputo:** El modelo YOLO26n debe ser de variante "Nano" para permitir su ejecución en servidores de nivel gratuito o bajo costo, manteniendo una latencia de respuesta mínima.³
- **Plataforma:** El despliegue inicial está restringido exclusivamente al sistema operativo Android, dada su cuota de mercado y la facilidad de integración con librerías de Google Jetpack.³
- **Conectividad:** El sistema requiere conexión de datos activa para la inferencia de visión artificial en la nube. El historial de diagnósticos previos está disponible offline mediante un cache local implementado con la librería Room en Android.³
- **Seguridad y Ética:** El sistema debe cumplir con la Ley 20.393 de responsabilidad penal de las personas jurídicas y las políticas de integridad de Duoc UC, garantizando el manejo ético de los datos del usuario.¹⁷

2.5. Suposiciones y Dependencias

- Se asume que la mayoría de los dispositivos de los usuarios cuentan con cámaras de al menos 8 megapíxeles y autoenfoco.³
- La precisión del modelo depende de la calidad del dataset de entrenamiento. El dataset utilizado cuenta con 31.940 imágenes distribuidas en 12 clases (4 frutas × 3 estados de madurez), recopiladas desde fuentes públicas: Kaggle (plátano, tomate, mango) y Mendeley (aguacate Hass, 14.710 imágenes con etiquetas de 5 etapas de madurez). El split es 70% entrenamiento / 15% validación / 15% test.³
- Se depende de la estabilidad de los servicios de infraestructura de AWS o Render para el mantenimiento de la disponibilidad del 95% durante la fase de evaluación.³

3. Requisitos Específicos

3.1. Interfaces Externas

3.1.1. Interfaz de Usuario (UI)

La aplicación móvil presentará una arquitectura visual centrada en el visor de cámara (Viewfinder). La pantalla principal incluirá un botón de captura prominente y un selector de tipo de fruta (aunque la IA intentará la detección automática). Tras el escaneo, la UI transicionará a una vista de resultados que utiliza colores semafóricos claros:

- **Verde:** Inmaduro (Requiere tiempo adicional).
- **Amarillo/Dorado:** Maduro (Punto óptimo para consumo).
- **Rojo/Oscuro:** Sobre-maduro (Consumir inmediatamente o procesar).³

3.1.2. Interfaz de Software

La comunicación entre el cliente Android y el servidor de Python se realizará mediante una API REST sobre HTTPS. Los endpoints principales serán:

- **POST /v1/predict:** Recibe una imagen en formato binario (multipart/form-data) y devuelve un objeto JSON con la clasificación, confianza y recomendaciones.³
- **GET /v1/history:** Recupera el historial de escaneos del usuario almacenado en PostgreSQL (producción) o SQLite (desarrollo), con soporte de paginación (parámetros limit y offset).³

3.1.3. Interfaz de Comunicaciones

El sistema requiere el uso del protocolo TLS 1.2 o superior para cifrar todas las comunicaciones. Se utilizará el formato JSON para el intercambio de datos estructurados, garantizando la interoperabilidad entre el frontend en Kotlin y el backend en Python.³

3.2. Funciones de Software Detalladas

Siguiendo la metodología de descomposición funcional, se definen los siguientes Requisitos Funcionales (RF):

ID	Requisito Funcional	Descripción Técnica
RF01	Captura y Normalización	El sistema permitirá capturar imágenes y las redimensionará automáticamente a 640x640 píxeles para ser compatibles con YOLO26n. ³
RF02	Inferencia de Visión Artificial	El backend ejecutará la red neuronal para identificar la especie de fruta y su estado de madurez con una confianza mínima configurable. ³
RF03	Persistencia de Diagnóstico	El sistema guardará la fecha, imagen (opcional), resultado y metadatos del escaneo en la base de datos relacional para auditoría. ³
RF04	Gestión de Recomendaciones	El sistema consultará una base de conocimientos para devolver consejos específicos según la fruta y su estado detectado. ³
RF05	Detección de Errores Ópticos	El sistema notificará al usuario si la imagen está demasiado oscura, borrosa o si no se detecta ninguna de las frutas soportadas. ³

3.3. Requisitos de Rendimiento

La latencia y la precisión son los indicadores de éxito (KPI) más críticos para MaduraApp. Un sistema lento en un supermercado será abandonado rápidamente por el usuario.³

- **Tiempo de Respuesta Extremo a Extremo:** El ciclo total (Captura → Envío → Inferencia → Respuesta → Visualización) no debe exceder los 5 segundos en redes 4G estándar.³
- **Precisión del Modelo (Accuracy):** Se exige un acierto mínimo del 75% en las pruebas de validación con imágenes no vistas previamente por el modelo.³
- **Concurrencia del Servidor:** El backend debe ser capaz de procesar al menos 5 peticiones simultáneas sin degradar el tiempo de respuesta en más de un 20%.³

3.4. Atributos del Sistema

3.4.1. Seguridad y Privacidad

Siguiendo los lineamientos de Duoc UC, la seguridad de la información se centra en tres pilares: Confidencialidad, Integridad y Disponibilidad.¹⁸

- **Cifrado:** Todas las imágenes enviadas a la nube deben viajar por canales cifrados con HTTPS para evitar interceptaciones (Ataques Man-in-the-Middle).²⁷
- **Autenticación:** El acceso a los datos de historial estará protegido por tokens de acceso básicos, asegurando que un usuario solo pueda ver sus propios registros.³
- **Auditoría:** Se mantendrán logs de acceso al servidor para identificar posibles intentos de uso no autorizado de los recursos cloud.¹⁷

3.4.2. Fiabilidad y Mantenibilidad

- **Confiabilidad:** El sistema debe operar correctamente bajo condiciones variables de red, manejando errores de timeout de forma elegante sin colapsar la aplicación móvil.³¹
- **Mantenibilidad:** El código backend debe seguir los principios de "FastAPI Best Practices", separando las rutas, esquemas de Pydantic y lógica de servicios para facilitar actualizaciones del modelo de IA sin afectar la API.⁷

4. Análisis Biológico de Frutos Climáticos

La eficacia de MaduraApp reside en su capacidad para interpretar digitalmente las señales biológicas de maduración de los frutos climáticos. A diferencia de los frutos no climáticos (como la piña o los cítricos, que no maduran tras ser separados de la planta y solo se deterioran), los frutos climáticos como el aguacate, el plátano, el tomate y el mango muestran cambios visuales predecibles impulsados por el etileno.³

4.1. El Aguacate (*Persea americana* - Variedad Hass)

El aguacate Hass es el candidato ideal para la visión artificial debido a su dramático cambio de color. Durante la maduración post-cosecha, la piel pasa de un verde brillante (Inmaduro) a un púrpura oscuro y finalmente a un negro intenso (Sobre-maduro).¹²

- **Mecanismo de Análisis:** MaduraApp utiliza redes neuronales para detectar no solo el tono (Hue) sino también la textura. Los aguacates inmaduros tienen una piel más lisa, mientras que los maduros presentan una textura rugosa característica causada por cambios en la turgencia celular.¹²
- **Métrica de Calidad:** La firmeza disminuye inversamente al avance del color. El sistema mapea el color detectado a una escala de firmeza estimada.³³

4.2. El Plátano (*Musa sapientum*)

La maduración del plátano es quizás el proceso más estudiado en la industria alimentaria. El cambio visual es impulsado por la degradación de la clorofila y la conversión de almidón en azúcares libres.¹⁴

- **Indicadores Visuales:** El sistema analiza el Índice de Color (CI). El paso del estado verde (CI 1) al amarillo con manchas marrones (CI 6) permite determinar el pico de sabor y textura.³⁴
- **Detección de Adulterantes:** Existe una diferencia visual sutil entre el plátano madurado naturalmente y el tratado con carburo de calcio (químico prohibido en algunas regiones por seguridad alimentaria). MaduraApp puede ser entrenada para detectar anomalías cromáticas asociadas a maduración química acelerada.³⁴

4.3. El Tomate (*Solanum lycopersicum*)

La USDA define 6 etapas de maduración para el tomate basadas en el color externo de la superficie. MaduraApp simplifica estas etapas para el consumidor pero mantiene la precisión técnica en el backend.⁴

Etapa USDA	Descripción Visual	Acción del Sistema
Green	100% verde. Muy firme.	Diagnóstico: "Inmaduro" (Espera 5-7 días)
Breaker / Turning	10-30% de color distinto al verde.	Diagnóstico: "Próximo a madurez" (Espera 2-3 días)
Pink / Light Red	30-90% de color rosa o rojo.	Diagnóstico: "Óptimo" (Consumo inmediato)
Red	Más del 90% rojo.	Diagnóstico: "Muy Maduro" (Consumir hoy)

El análisis mediante visión artificial en tomates es particularmente valioso porque permite una evaluación no destructiva que supera la subjetividad humana, especialmente en condiciones de iluminación variable en mercados abiertos.¹³

4.4. El Mango (*Mangifera indica*)

El mango presenta el mayor desafío para la visión artificial, ya que el color rojo de la piel no siempre es un indicador de madurez, sino de exposición solar del fruto en el árbol.³⁷

- **Foco del Análisis:** MaduraApp se entrena para priorizar la morfología (forma de los "hombros" en la base del tallo) y el cambio de color de la pulpa interna que se trasluce en la piel (amarilleamiento) más que el rubor rojo superficial.³⁷
- **Correlación Cromática:** Se utilizan espacios de color como CIELAB o HSV para detectar cambios sutiles en la saturación del amarillo, correlacionándolos con la disminución de la acidez titulable y el aumento de los sólidos solubles.³⁵

5. Arquitectura Tecnológica y Cómputo en la Nube

5.1. El Cliente Móvil: Android Nativo con Kotlin

La elección de un desarrollo nativo responde a la necesidad de manipular el hardware de la cámara con precisión milimétrica. La librería CameraX permite implementar un ImageAnalysis use case que proporciona buffers de imagen accesibles por CPU para realizar pre-procesamiento local antes del envío a la nube.⁶

- **Gestión de Ciclo de Vida:** Kotlin permite vincular los procesos de cámara al Lifecycle de la Activity de Android, asegurando que los recursos se liberen cuando la app pase a segundo plano, optimizando el consumo de batería.⁶
- **Comunicación Asíncrona:** Se utilizan Corrutinas de Kotlin para manejar las peticiones de red hacia la API de Python, evitando bloqueos en la interfaz de usuario (ANR - App Not Responding) mientras se espera el diagnóstico de la IA.⁷

5.2. El Backend: Python y FastAPI

FastAPI se ha seleccionado por su capacidad de manejar peticiones asíncronas de forma nativa, lo cual es vital cuando se tiene un modelo de IA que puede tardar varios milisegundos en procesar una imagen.⁸

- **Validación de Datos:** Utiliza modelos de Pydantic para garantizar que los datos recibidos del móvil (como el tipo de fruta o coordenadas) cumplan con el esquema esperado antes de llegar a la lógica de inferencia.⁷
- **Integración de ML:** El backend carga el modelo YOLO26n utilizando la librería Ultralytics, implementando una importación diferida (lazy import) para evitar que el módulo bloquee el inicio del servidor cuando los pesos no están disponibles.⁸

5.3. Estrategia Cloud: IaaS vs PaaS

Para un proyecto con las características de MaduraApp, se ha analizado la mejor opción de despliegue en la nube, comparando flexibilidad y costo.¹⁰

Modelo	Justificación Técnica para MaduraApp
IaaS (AWS EC2)	Ofrece control total sobre el sistema operativo para optimizaciones de bajo nivel de la GPU/CPU. Recomendado para tráfico alto y constante. ¹⁰
PaaS (Render / AWS Lambda)	Ideal para el MVP. Permite enfocarse en el código de Python sin gestionar servidores. AWS Lambda es excelente para tráfico esporádico pero sufre de latencia inicial (cold start) que podría afectar el requisito de los 5 segundos. ¹⁰

Decisión Arquitectónica: Para el MVP académico se utilizará un servicio PaaS de nivel gratuito (Render o AWS App Runner), que gestiona el despliegue automático del contenedor Docker sin administración de servidores. Esta elección prioriza la velocidad de despliegue y el costo cero sobre el control de bajo nivel, adecuado para el nivel de tráfico esperado durante la evaluación. La migración a AWS ECS o SageMaker se contempla para fases de escalado post-MVP.²²

6. Análisis de Causa Raíz: Reducción del Desperdicio Alimentario

Para que MaduraApp sea una solución efectiva, su diseño debe abordar las causas fundamentales del desperdicio en la cadena de suministro. El uso del Diagrama de Ishikawa (Espina de Pescado) permite identificar estas causas en el sector retail y doméstico.⁴²

6.1. Diagrama de Ishikawa: Causas del Desperdicio de Frutas Frescas

Categoría (6M)	Causa Identificada	Impacto en el Desperdicio
Manpower (Mano de obra)	Falta de capacitación del personal de tienda en manejo de climatéricos. ²	Manipulación ruda que causa daños físicos y maduración prematura.
Methods (Métodos)	Estimación visual subjetiva del estado de madurez. ¹⁴	Selección errónea que lleva a productos sobre-maduros en estantería.
Machines (Máquinas)	Fallas en la cadena de frío o falta de infraestructura de almacenamiento. ¹	Aceleración descontrolada de la maduración post-cosecha.
Materials (Materiales)	Variabilidad estacional en la calidad de la fruta entrante. ⁴⁵	Dificultad para estandarizar los tiempos de venta.
Measurement (Medición)	Falta de herramientas tecnológicas no invasivas en el punto de venta. ¹	El cliente "presiona" la fruta para ver si está madura, dañándola.
Mother Nature (Entorno)	Temperaturas ambientales extremas en mercados abiertos. ²	Degradación acelerada de la calidad organoléptica.

Solución MaduraApp: Al introducir una herramienta de Medición objetiva y no invasiva, el sistema elimina la necesidad de que el cliente dañe físicamente el producto y proporciona un Método estándar que reduce el error humano en la selección.²

6.2. Árbol de Oportunidades y Soluciones

El diseño de MaduraApp se alinea con las oportunidades identificadas para mejorar la sostenibilidad del sistema alimentario.⁴⁶

- **Resultado Deseado:** Reducción del 15% en el desperdicio de frutas climatéricas en el punto de venta en el primer año de implementación.³
- **Oportunidad 1:** Empoderar al consumidor con datos técnicos transparentes.³ Solución: Aplicación móvil de diagnóstico instantáneo.
- **Oportunidad 2:** Optimizar la logística inversa del retail basado en estados de madurez reales.²³ Solución: Panel de administración (Dashboard) con mapa de calor de calidad de productos por zona.

7. Metodología y Gestión del Proyecto

El desarrollo de una solución que integra hardware (sensores de cámara), software distribuido e inteligencia artificial requiere un marco de trabajo que combine la agilidad del desarrollo de software con el rigor del análisis de datos.¹⁹

7.1. Integración de Agile Scrum y CRISP-DM

MaduraApp adopta una metodología híbrida para gestionar la incertidumbre técnica del modelo de IA:

- **Agile Scrum:** Utilizado para el desarrollo del frontend Android y el backend FastAPI. Los Sprints de 2 semanas permiten entregas parciales de la interfaz y la integración de APIs.³
- **CRISP-DM (Cross-Industry Standard Process for Data Mining):** Aplicado específicamente al ciclo de vida del modelo de IA. Este proceso cíclico asegura que la preparación de datos (que toma el 60-80% del tiempo) y la evaluación de la precisión se realicen de forma sistemática antes de cada despliegue.²⁰

7.2. Fases del Proyecto (Cronograma de 16 semanas)

Semana s	Fase	Entregables Clave	Estado
1-4	Comprensión de Negocio y Datos	Dataset etiquetado (31.940 imgs, 4 frutas climatéricas, Kaggle + Mendeley). ³	[OK] Completado
5-8	Modelado y Desarrollo Backend	Modelo YOLO26n en entrenamiento; API REST funcional en Python con FastAPI, SQLAlchemy, Alembic y 9 tests reales. ³	[OK] Completado
9-12	Desarrollo Frontend y Conectividad	App Android con CameraX, MVVM, historial con Room, 20 tests JVM. ³	[OK] Completado
13-16	Pruebas, Ajuste y Despliegue Cloud	Validación $mAP@50 \geq 0.75$; despliegue en Render/AWS App Runner. ³	[~] En progreso

8. Consideraciones Éticas y Cumplimiento Normativo

MaduraApp se compromete con los estándares de integridad institucional de Duoc UC. La aplicación de la Ley 20.393 implica que el desarrollo debe prevenir delitos informáticos y garantizar el resguardo de activos de información.¹⁷

- **Transparencia:** Los usuarios deben ser informados de que las imágenes capturadas se procesan en la nube únicamente con fines de análisis de madurez.³
- **Inclusión:** La interfaz debe diseñarse para ser accesible, utilizando contrastes de color adecuados para personas con daltonismo, asegurando que el "semáforo de madurez" sea interpretable mediante símbolos además de colores.³
- **Sostenibilidad:** Al reducir el desperdicio, el proyecto contribuye directamente a los objetivos de desarrollo sostenible (ODS) relacionados con el consumo y la producción responsables.¹

9. Conclusiones

La Especificación de Requisitos de Software para MaduraApp demuestra que la convergencia de la visión artificial y la computación móvil nativa es una vía tecnológicamente viable y socialmente necesaria para combatir la ineficiencia alimentaria.² La integración de un frontend robusto en Kotlin con un backend altamente escalable en Python permite superar las limitaciones de hardware de los dispositivos tradicionales, democratizando el acceso a herramientas de agronomía de precisión para el consumidor común.³

La rigurosa definición de requisitos bajo el estándar IEEE 830 asegura que el desarrollo del MVP sea trazable y que los atributos de calidad como la seguridad y la latencia no sean sacrificados en favor de la funcionalidad.³ MaduraApp representa, por tanto, un modelo de cómo la tecnología "Cloud-Native" puede transformar procesos biológicos complejos en soluciones de software simples, intuitivas y de alto impacto ambiental.¹

Obras citadas

- 1 Food loss and waste in the food supply chain - FAO Knowledge Repository.
- 2 An exploratory study of possible food waste risks in supermarket fruit and vegetable sections - SciELO.
- 3 Documento MVP AgroVision AI.pdf
- 4 Comparative Analysis of R-CNN and YOLOv8 Segmentation Features for Tomato Ripening Stage Classification and Quality Estimation - MDPI.
- 5 Machine Learning for Automatic Classification of Tomato Ripening Stages Using Electrophysiological Recordings - Frontiers.
- 6 CameraX architecture | Android media.
- 7 FastAPI Best Practices - Auth0.
- 8 FastAPI Best Practices: A Complete Guide for Building Production-Ready APIs - Medium.
- 9 Comparison of Cloud-Computing Providers for Deployment of Object-Detection Deep Learning Models - MDPI.
- 10 SaaS vs PaaS vs IaaS — What's the Difference? - Liferay DXP.
- 11 Banana Mango Ripeness Detection Object Detection Model - Roboflow Universe.
- 12 Computer Vision for Avocado Ripeness Detection - Nick Gentz.
- 13 Computer Vision-based Detection Models for Classifying Ripening Stages of Tomato Fruits - JISEM.
- 14 Automatic Classification of the Ripeness Stage of Mango Fruit Using a Machine Learning Approach - ResearchGate.
- 15 Best Practices in FastAPI Architecture - Zyneto.
- 16 Machine Learning With CameraX and MLKit - Medium.
- 17 Manual del Sistema de Ética e Integridad (SEI) - Duoc UC.
- 18 Política General de Seguridad de la Información - Duoc UC.
- 19 Scaling Data Science: How We Use CRISP-DM and Agile - Motivus.
- 20 Leveraging XP and CRISP-DM for Agile Data Science Projects - arXiv.
- 21 AIoT-Enabled Data Management for Smart Agriculture - IEEE Xplore.
- 22 Deploying ML Models to Production: AWS Lambda vs ECS vs EKS - DEV Community.
- 23 Retail - Why and How to Measure Food Loss and Waste - CEC.
- 24 Retail Food Wastage - ResearchGate.
- 25 Cost Saving Strategies for Deploying Models - New Math Data.
- 26 Fruit-Ripeness-Dataset - Kaggle.
- 27 Servicios Digitales Docentes - Seguridad de la información - Duoc UC.
- 28 Image analysis | Android media - Android Developers.
- 29 Principios Básicos de Seguridad de la Información (CIA) - Auditool.
- 30 8 maneras de garantizar la integridad de los datos - Vaisala.
- 31 Software Quality Attributes in Software Engineering.
- 32 Essential Software Quality Attributes Explained - Arobit.
- 33 The Synthetic Avocado Ripeness Dataset - Medium.
- 34 Ripen banana dataset - PMC.

- 35 Determination of Fruit Ripeness Degree of 'Carabao' Mango - Philippine Journal of Science.
- 36 Classification of tomato ripeness in the agricultural industry using a computer vision system.
- 37 Maturity & Ripeness Guide - Mango.org.
- 38 A Grading Method for Mangoes on the Basis of Peel Color Measurement Using a Computer Vision System - SCIRP.
- 39 Automatic Classification of the Ripeness Stage of Mango Fruit Using a Machine Learning Approach - MDPI.
- 40 Comparative Analysis of AWS Model Deployment Services - arXiv.
- 41 Comparing SageMaker and Containerized Lambda for ML Predictions - Medium.
- 42 What is a Fishbone Diagram? Ishikawa Cause & Effect Diagram - ASQ.
- 43 Understanding the Ishikawa Diagram - Kaizen Institute.
- 44 Machine learning classification of mango maturity based on carotene content from Raman spectra - PMC.
- 45 Economic Drivers of Food Loss at the Farm and Pre-Retail Sectors - USDA ERS.
- 46 FREE Opportunity Solution Tree Template - Miro.
- 47 Guía de uso para aplicar el árbol de soluciones - FAO.
- 48 Making the right choice: agile approaches that can be used in data science projects - Adesso.
- 49 CRISP-DM vs. Agile — Complimentary PM Frameworks - Medium.
- 50 The evolution of CRISP-DM for Data Science - Tilburg University.